

UNIVERSIDAD NACIONAL DE LA PATAGONIA AUSTRAL
Unidad Académica Río Gallegos



Estructura de Datos

Tema 3: Búsqueda

Docente: Mg. Héctor Reinaga

Indice



- Introducción
- Algoritmos básicos de búsqueda
 - Secuencial
 - Binaria
- Algoritmos avanzados
 - Interpolación
 - Fibonacci
- Hashing
 - Introducción
 - Métodos de transformación de claves
 - Restas sucesivas
 - Método de división o resto
 - Método del medio cuadrado
 - Truncamiento
 - Método de superposición
 - Soluciones al problema de colisiones
 - Rehashing
 - Prueba lineal o secuencial
 - Agrupamiento o clustering
 - Prueba cuadrática
 - Doble dirección hash
 - Arreglos anidados o cubos
 - Encadenamiento o tablas hash abiertas
 - Zona de desbordamiento

Introducción

- Buscar en un arreglo un determinado valor, significa localizar un elemento del arreglo cuyo contenido coincida con él.
- Una vez finalizada la búsqueda puede suceder:
 - Con éxito: habiendo localizado la posición donde estaba almacenado el elemento buscado, o
 - Sin éxito: que la búsqueda no haya tenido éxito, concluyéndose que no existía ningún elemento a buscar.
- Las búsquedas son independientes del estado del arreglo.
- Se puede buscar en arreglos ordenados como en arreglos desordenados, utilizando diferentes algoritmos de búsqueda.

Algoritmos básicos

Búsqueda

Búsqueda Secuencial

- Es la forma más simple de buscar un elemento y consiste en examinar secuencialmente uno a uno hasta encontrar el buscado o haber revisado todos los elementos sin éxito.
- Implementación:

```
public static int sequentialSearch(int[] vec, int search) {
    int pos=0;
    while(pos<vec.length) {
        if(vec[pos] == search)
            return pos;
        pos++;
    }
    return -1;
}
```

- Puede mejorarse mediante un centinela.

Búsqueda Secuencial: complejidad

- **MEJOR CASO:**
 - El elemento buscado se encuentra en la primera posición examinada.
 - Por tanto, su complejidad será **$O(1)$** .
- **PEOR CASO:**
 - Sucede cuando el elemento buscado se encuentra al final del arreglo.
 - Se requieren n ejecuciones del bucle *mientras*.
 - Por lo tanto, la cantidad de tiempo es de la forma $an + b$ para ciertas constantes a y b . En notación O , $O(an+b) = O(an) = O(n)$.
- **CASO MEDIO:**
 - Cada elemento almacenado tiene la misma probabilidad de ser buscado.
 - La media se puede calcular tomando el tiempo total de encontrar todos los elementos y dividiéndolo por n :
 - Total = $a(1 + 2 + \dots + n) + bn = a(n(n+1)/2) + bn$
 - Media = (Total / n) = $a((n+1)/2) + b$ que es **$O(n)$** .

Búsqueda Binaria

- Si los elementos sobre los que se realiza la búsqueda están ordenados, entonces podemos utilizar la búsqueda binaria.
- **La idea básica de la búsqueda binaria o dicotómica es reducir el tamaño del problema a la mitad en cada iteración.**
- Se basa en comparar el elemento a buscar con el elemento que se encuentra en la mitad del arreglo, entonces:
 - Si es **igual**, finaliza con éxito la búsqueda ya que se ha encontrado el elemento.
 - Si el elemento buscado es **menor** que el elemento medio, entonces el elemento está en la mitad inferior de la tabla.
 - Si es **mayor**, el elemento está en la mitad superior.

Búsqueda Binaria

- La implementación de este algoritmo, se muestra a continuación:

```
public static int binarySearch(int[] vec, int search) {
    int first=0;
    int mid;
    int last=vec.length - 1;
    while(first<=last) {
        mid=(first + last) / 2;
        if(vec[mid] == search)
            return mid;
        else
            if(vec[mid]>search)
                last=mid - 1;
            else
                first=mid + 1;
    }
    return -1;
}
```

Búsqueda Binaria: complejidad

- **CASO ÓPTIMO:**
 - La búsqueda binaria requiere sólo una comparación (elemento buscado en el centro).
 - Su tiempo de ejecución óptimo no depende de la cantidad de datos: es constante y por tanto proporcional a 1, es decir, **$O(1)$** .
- **PEOR y CASO MEDIO:**
 - En el peor caso sí dependen de N. La búsqueda binaria divide el arreglo, requiriendo un tiempo **$O(\log n)$** . Lo mismo ocurre para el caso medio.

Algoritmos avanzados

Búsqueda

Búsqueda por interpolación

- Parecido a la búsqueda binaria.
 - Requisito: elementos ordenados
 - El algoritmo de búsqueda binaria siempre selecciona el elemento central del vector para compararlo con el elemento a buscar y dividir el vector.
- En este tipo de búsqueda, el arreglo se va dividiendo acorde a una estimación.
 - El elemento no se busca a partir del elemento central
 - sino aquel que se “correspondería” con el elemento buscado, si la distribución de valores en el vector fuera **uniforme**.
 - Se delimita por medio de los valores resultantes de la interpolación.
 - Se aplica una formula de interpolación

Búsqueda por interpolación

- La implementación de este algoritmo, se muestra a continuación:

```
public static int interpolationSearch(int[] vec, int search) {
    int first=0;
    int mid;
    int last=vec.length - 1;
    while(search>vec[first] & search<vec[last]) {
        mid=first + (int)Math.abs(Math.floor((search - vec[first]) * (last-first) / (vec[last]-vec[first])));
        if(search==vec[mid])
            return mid;
        else
            if(search<vec[mid])
                last=mid - 1;
            else
                first=mid + 1;
    }
    return -1;
}
```

Búsqueda por interpolación

- Este tipo de búsqueda puede ser mejor que la búsqueda Binaria, si:
 - Cada acceso es muy costoso
 - Por ejemplo, el arreglo se encuentra almacenado en el disco y cada comparación requiere un acceso a disco.
 - Los datos se encuentran ordenados y medianamente distribuidos uniformemente
 - Por ejemplo: las páginas de un libro, una guía telefónica cumplen esas características
 - No este caso: [1, 2, 3, 4, 5, 6, 7, 8, 16, 32, 355, 1000, 12300...].
- Ejemplo:
 - Un arreglo de 1000 elementos.
 - El elemento más pequeño del arreglo es 1.000.
 - El elemento más grande del arreglo es 1.000.000.
 - Se busca el elemento con el valor 12.000.
 - Luego se espera encontrar el elemento, en la posición 11 aproximadamente (siempre asumiendo que los elementos están uniformemente distribuidos).

Búsqueda por interpolación

- Ejemplo:

n = 1.000	vec	1.000		1.000.000
search = 12.000		0	?	999
		↑	↑	↑
		first	mid	last

- Búsqueda Binaria:

$$mid = \frac{(first + last)}{2} \Rightarrow mid = \frac{(0 + 999)}{2} = 500$$

- Búsqueda por Interpolación:

$$mid = first + \frac{(search - vec[first]) * (last - first)}{vec[last] - vec[first]}$$

$$mid = 0 + \frac{(12.000 - 1.000) * (999 - 0)}{1.000.000 - 1.000} = \frac{10.989.000}{999.000} = 11$$

Búsqueda por interpolación: complejidad

- Peor caso:
 - Cuando los números no se encuentran uniformemente distribuidos, el tiempo de ejecución puede ser lineal y todos los elementos podrían ser examinados.
- Mejor caso:
 - Si los elementos son de manera razonable uniformemente distribuidos, el tiempo de ejecución ha sido demostrado como **$O(\log \log n)$** (aplicar el logaritmo dos veces en la sucesión).
 - Por ejemplo, para $n=4$ billones, $\log n$ es alrededor 32 y $\log \log n$ es aproximadamente 5.

Búsqueda Fibonacci

- Similar a la búsqueda binaria.
- En vez de dividir el arreglo por la mitad, divide el arreglo según los números de Fibonacci.
- Los números de Fibonacci se definen de la siguiente manera:
 - $F(0) = 0$
 - $F(1) = 1$
 - $F(n) = F(n - 1) + F(n - 2)$ (para $n \geq 2$)

F	F(N)
1	0
2	1
3	1
4	2
5	3
6	5
7	8
8	13
9	21
10	34
11	55
12	89
22	10946
52	2.0E10
102	5.7E20

Búsqueda Fibonacci (algoritmo)

```
//el arreglo debe estar cargado con los números de Fibonacci
public static int fibonacciSearch(int[] vec, int search) {
    int f1=1;
    int f2=1;
    int faux;
    int mid;
    int length=vec.length - 1;
    while (f1<length) { //encuentra a f1 tal que f1>=length
        f1=f1 + f2; //siguiente número Fibonacci
        f2=f1 - f2; //guarda al f1 anterior
    }
    f1=f1 - f2; //encuentra el número Fibonacci más chico
    f2=f2 - f1; //f1=fib(j-2), f2=fib(j-3)
    mid=length - f1 + 1;
    while (search!=vec[mid]) { //si no lo encuentra
        if (mid<0 | search>vec[mid]) { //busca en la mitad más baja
            if (f1==1) //si no lo encuentra retorna -1
                return -1;
            mid=mid + f2; //disminuye los números de Fibonacci
            f1=f1 - f2;
            f2=f2 - f1;
        }
        else { //busca en la mitad más alta
            if (f2==0) //si no lo encuentra retorna -1
                return -1;
            mid=mid - f2; //disminuye los números de Fibonacci
            faux=f1 - f2; //esta vez, disminuye mas
            f1=f2; //para el arreglo más chico
            f2=faux;
        }
    }
    return mid;
}
```

Búsqueda Fibonacci

- Concepto matemático:
 - Este método es utilizado para obtener un punto óptimo en funciones no diferenciables sin utilizar derivadas. Este método es eficiente para aproximar, bajo cierto margen de error, un punto máximo o mínimo en funciones de una sola variable. Con este método se conoce ya el rango inicial de búsqueda y en cada evaluación el método tiende a acorralar el punto óptimo.
- El algoritmo de búsqueda de Fibonacci divide el rango de búsqueda en dos partes, de acuerdo a los números de Fibonacci, luego se compara el elemento a buscar con la posición $F(n - 2)$. Si el elemento es más chico, se continua en la mitad más baja del arreglo, y en caso contrario, con la mitad más alta del arreglo.
- Al igual que la búsqueda Binaria, en el peor caso, el tiempo de ejecución de la búsqueda Fibonacci es de **$O(\log n)$** .

Búsquedas avanzadas

Hashing

Introducción

- El origen de los algoritmos de hash, es la ambición de los científicos por encontrar una forma mas rápida de encontrar la información.

Tipo de búsqueda	Complejidad
Búsqueda secuencial	$O(N)$
Arbol B	$O(\log N)$
???	$O(1)$

- El concepto de Hash es un caso ideal en la recuperación de la información $O(1)$
- Una función Hash es una caja negra que tiene como entrada una llave (clave) y como una salida una dirección.

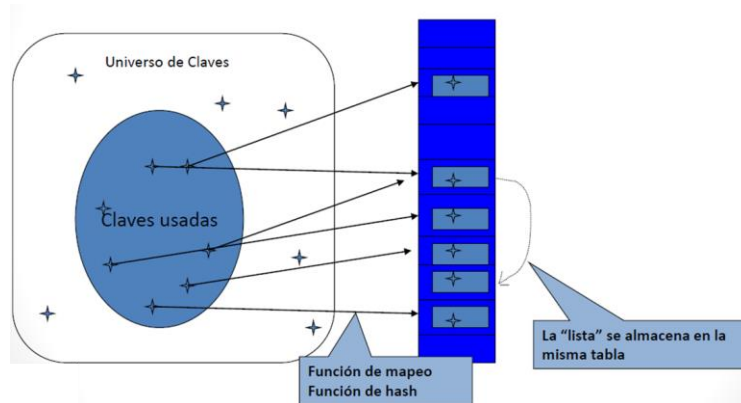
Hashing

- Es un método que aumenta la velocidad de búsqueda
 - los datos no necesitan estar ordenados
 - y es independiente del número n .
- **El hashing consiste en convertir el elemento almacenado (numérico o alfanumérico) en una dirección (índice) dentro del arreglo.**
- Una buena función Hash debería:
 - Distribuir las claves de manera uniforme en la tabla de *hash* (**hash uniforme**).
 - Ser fácil y rápida de calcular.
 - Producir tan pocas colisiones como sea posible.
 - La función de hash ideal debería ser biyectiva.
- Desventaja:
 - La función de *hash* puede producir el mismo resultado para dos claves diferentes (**colisión**).

Estrategias de resolución de colisiones

- Hash cerrado
 - se utiliza cuando el número máximo de elementos que se va almacenar es conocida, y se puede crear desde el inicio una tabla del tamaño específico a utilizar
 - almacena las claves en las mismas entradas del arreglo o tabla hash
- Hash abierto
 - crear una lista asociada a cada entrada del arreglo

Hash cerrado

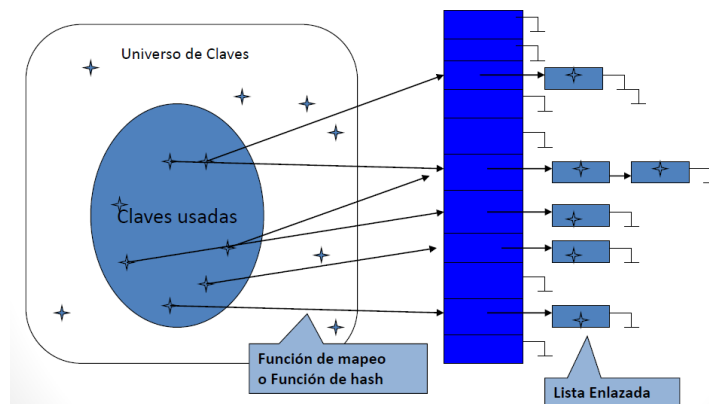


Algoritmos de Búsqueda

Estructura de Datos

23

Hash abierto



Algoritmos de Búsqueda

Estructura de Datos

24

Ejemplo de uso de un función hash

- El código ASCII asigna un número a cada letra o signo de puntuación.

65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/

Ejemplo Hash Cerrado

0	"Camila"
1	
2	
3	"Juan"
4	"Pedro"
5	"Maria"
6	"Martin"
7	
8	
9	"Daniela"

FHash("Juan") = 3

FHash("Maria") = 5

FHash("Pedro") = 3

FHash("Martin") = 4

FHash("Daniela") = 9

FHash("Camila") = 9

Que pasa si se elimina a Juan?

Hash cerrado

- Se necesita saber si en ese lugar se borro algo.
- Cada celda necesita:
 - Una clave
 - Un valor
 - Un estado (vacío, ocupado, borrado)

Hash cerrado: borrados

0	"O"	"Camila"
1	"V"	
2	"V"	
3	"B"	
4	"O"	"Pedro"
5	"O"	"Maria"
6	"O"	"Martin"
7	"O"	
8	"V"	
9	"V"	"Daniela"

FHash("Juan") = 3

FHash("Maria") = 5

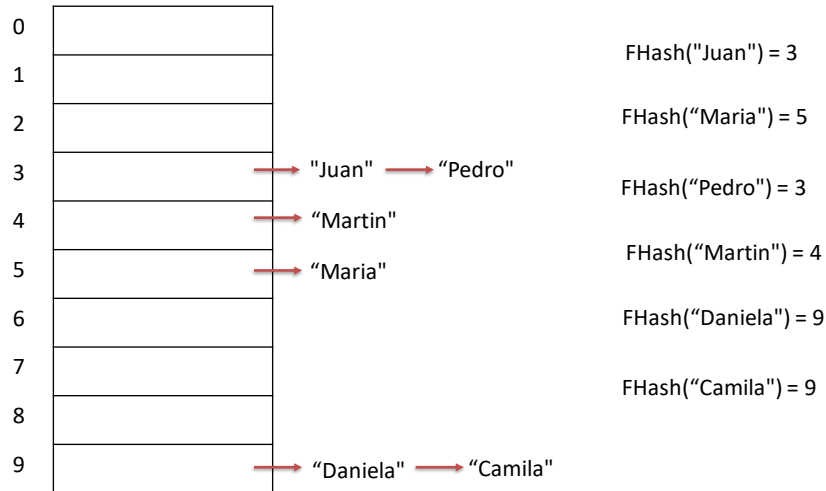
FHash("Pedro") = 3

FHash("Martin") = 4

FHash("Daniela") = 9

FHash("Camila") = 9

Ejemplo Hash Abierto



Algoritmos de Búsqueda

Estructura de Datos

29

Factor de carga

- Se define el factor de carga de una tabla como:
 - $\alpha = n / M$
- Siendo n la cantidad reales de elementos de la tabla, y M la capacidad o tamaño de la tabla.

Algoritmos de Búsqueda

Estructura de Datos

30

Redimensión Hash Cerrado

- Una buena métrica es tener $\alpha < 0,7$
- Problema
 - el hashing cerrado se empieza a degradar a medida que empiezan a haber mas colisiones
- Ventaja
 - los datos se encuentran en una "zona cercana" a la posición indicada por la función de Hashing

Redimensión Hash Abierto

- A diferencia del cerrado, puede tener un factor de $a > 1$.
- Se deben hacer pruebas para determinar el mejor valor, pero rondando entre 2 y 3 suele ser una buena opción.

MÉTODOS DE TRANSFORMACIÓN DE CLAVES

Hashing

- Los métodos de transformación de claves (hashing) que vamos a ver son:
 - Restas sucesivas
 - Método de división o resto
 - Método del medio cuadrado
 - Truncamiento
 - Método de superposición

Restas sucesivas

- Se emplea para claves numéricas donde existen diferencias de tamaño conocido, obteniéndose direcciones consecutivas.
- Ejemplo:
 - Número de expediente de un alumno universitario, formado por:
 - año de entrada en la universidad + número identificativo de tres cifras
 - suponiendo que entran un máximo de 500 alumnos al año, se le asignarían las claves:

2019-001 --> 0 = (2019001 – 2019000) - 1

2019-002 --> 1 = (2019002 – 2019000) - 1

...

2019-499 --> 498 = (2019499 – 2019000) - 1

2019-500 --> 499 = (2019500 - 2019000) - 1

Método de división o resto

- Consiste en tomar el módulo de la división de la clave entre el número de componentes del arreglo.
- Se elige un número m menor o igual que la capacidad n de elementos.
- La función hash queda definida por la siguiente fórmula:
 - $H(K) = K \bmod N$

Método de división o resto

- Ejemplo:

$$13602499 \% 13 = 1$$



$$12345678 \% 13 = 7$$



Método del medio o mitad cuadrado

- Consiste en elevar al cuadrado la clave y tomar los dígitos centrales, y el resultado sería la dirección.
- Funcion hash:
 - $H(K) = \text{digitos_centrales}(K^2)$
- Ejemplo:
 - $123 * 123 = 15129 \rightarrow 51$
 - $136 * 136 = 18496 \rightarrow 84$
 - $730 * 730 = 532900 \rightarrow 29$

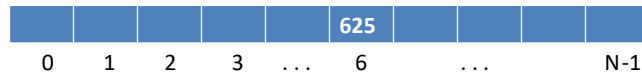
Método del medio o mitad cuadrado

- Ejemplo:

$$730^2 = 532900 \rightarrow 29$$



- $625^2 = 390625 \rightarrow 06$



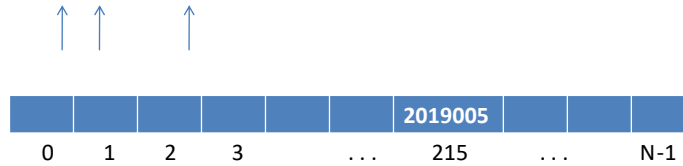
Truncamiento

- Consiste en tomar algunos dígitos de la clave y formar con ellos un índice.
- Funcion hash:
 - $H(K) = \text{elegirdigitos}(d_1, d_2 \dots d_n) + 1$

Truncamiento

- Ejemplo:

Clave 2019005  Clave = 215



Método de superposición

- Consiste en la división de la clave en partes iguales, y su combinación en un modo conveniente
 - utilizando suma o multiplicación para obtener el índice.

Método de superposición (1)

- Ejemplo:

- Superposición por desplazamiento* donde todas las partes se suman entre sí.

$$13000000 = 130 + 000 + 00 = 130$$



$$12345678 \rightarrow 123 + 456 + 78 = 657$$



Método de superposición (2)

- Ejemplo:

- Superposición por plegamiento* se hace la inversa a las partes pares X2, X4, ..., antes de sumarlas, con el fin de afinar más.

$$12345678 \rightarrow 855 = 123 + 654 + 78$$



$$71140205 \rightarrow 920 = 711 + 204 + 05$$



COLISIONES

Colisión

- Cuando 2 o mas claves dan como resultado el mismo valor en la funcion de hash.
- Para resolver las colisiones se utilizan los siguientes métodos:
 - Rehashing o Reasignación.
 - Arreglos anidados o Cubos
 - Encadenamiento o Tablas Hash Abiertas.
 - Zona de Desbordamiento.

Rehashing o Reasignación

- Se trata de insertar o buscar un elemento cuya posición ya está ocupada, en otra posición disponible en la tabla.
- Existen varios métodos que trabajan bajo el principio de comparación y reasignación de elementos:
 - Prueba lineal o secuencial
 - Agrupamiento o clustering
 - Prueba cuadrática
 - Doble dirección hash

Prueba lineal o secuencial

- Detectada la colisión, se recorre el arreglo secuencialmente a partir del punto de colisión, buscando el elemento o una posición vacía.
- Se trata el arreglo como una estructura circular.

Rehashing: Prueba lineal o secuencial.

- Ejemplo:

1	80	
2		
3		
4	43	
5	54	
6	25	
7	56	
8	35	
9	13	
10	104	

K	H(K)
25	6
43	4
56	7
35	6
54	5
13	4
80	1
104	5

Rehashing: Prueba lineal o secuencial

- Desventajas:
 - Existe fuerte agrupamiento alrededor de ciertas claves.
 - Otras zonas del arreglo pueden permanecer vacías.
 - A concentraciones de claves muy frecuentes, la búsqueda tenderá a ser secuencial.

Agrupamiento o Clustering.

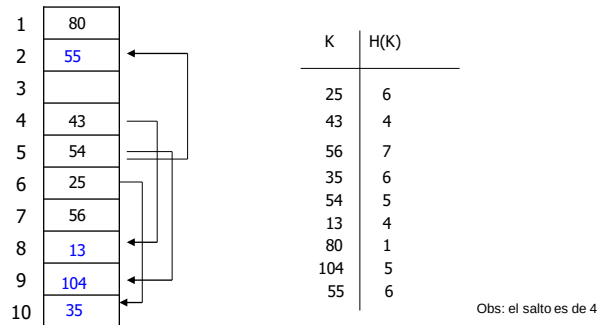
- **Agrupamiento o clustering**
 - ocurre cuando dos claves que inicialmente generan una asignación en dos sitios diferentes, luego compiten entre sí en reasignaciones sucesivas.
 - $H(X) \neq H(Y)$ pero $RH(X) = RH(Y)$ en reasignaciones sucesivas.
- Existen dos técnicas para minimizar el agrupamiento:
 - **Prueba cuadrática**
 - **Doble dirección hash**

Rehashing: Prueba Cuadrática

- Similar al método de Prueba Lineal o Secuencial.
- Este método genera las direcciones alternativas como:
 - $D + 1$
 - $D + 4$
 - $D + 9$
 - ...
 - $D + i^2$
- Desventaja:
 - Existe un agrupamiento alrededor de ciertas claves y casillas no visitadas.

Rehashing: Prueba Cuadrática

- Ejemplo:



Rehashing: Doble direccionamiento hash

- El método consiste en que cuando se detecta la colisión, se debe generar otra dirección aplicando la función hash H_2 a la dirección obtenida anteriormente.
- Se detiene el proceso cuando se encuentra el elemento, o cuando se encuentra una posición vacía.
- La función hash H_2 que se aplique puede ser diferente con respecto a la que se aplicó originalmente a la clave.

Rehashing: Doble direccionamiento hash

- Ejemplo:

	K	H(K)	H'(D)	H'(D')	H(D'')
1	80	6	---	---	---
2		4	---	---	---
3		7	---	---	---
4	43	6	8	---	---
5	54	5	---	---	---
6	25	4	6	8	10
7	56	1	---	---	---
8	35	5	7	9	---
9	104	5	---	---	---
10	13	---	---	---	---

Arreglos anidados o Cubos

- Este método consiste en que cada elemento del arreglo tenga otro arreglo en el cual se almacenen los elementos colisionados.
- Aunque la solución parece sencilla, es ineficiente.
- Surge un problema:
 - elegir el tamaño adecuado del arreglo que permita un equilibrio entre costo memoria y número de valores colisionados que puede almacenar.
- Cuando el cubo se llena, debemos tratar de nuevo con colisiones.

Arreglos anidados o Cubos

- Ejemplo:

	v						
1	80				...		
2							
3							
4	43	13					
5	54	104					
6	25	35					
7	56						
8							
9							
10							

k	H(k)
25	6
43	4
46	7
35	6
54	5
13	4
80	1
104	5

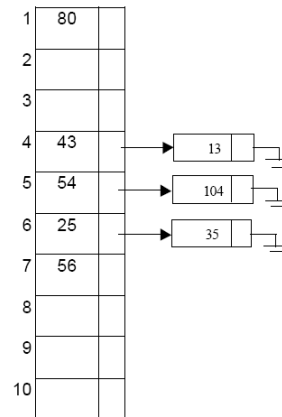
Encadenamiento o Tablas Hash Abiertas

- Es uno de los métodos mas eficiente debido al dinamismo propio de las listas.
- Consiste en que cada elemento del arreglo tenga un apuntador a una lista enlazada.
- Cualquiera que sea el número de colisiones, siempre se puede tratar una más.
- Desventaja:
 - Ocupa un espacio adicional que una tabla y requiere el manejo de listas enlazadas.
 - Si las listas crecen demasiado se pierde la facilidad de acceso directo del método hash.

Encadenamiento o Tablas Hash Abiertas

- Ejemplo:

K	H(K)
25	6
43	4
56	7
35	6
54	5
13	4
80	1
104	5



Encadenamiento o Tablas Hash Abiertas

Estas listas pueden reorganizarse para maximizar la eficiencia de búsqueda, utilizando diferentes métodos:

- **Probabilidad.**
- **Movimiento al frente.**
 - Cuando una búsqueda ha tenido éxito, el elemento encontrado es retirado de su posición actual en la lista y colocado en la cabeza de dicha lista.
- **Trasposición.**
 - Cuando una búsqueda de un elemento ha tenido éxito es intercambiado con el elemento que le precede inmediatamente, de manera que si es accedido muchas veces llegará a ocupar la primera posición de la lista.

Encadenamiento o Tablas Hash Abiertas

- Ventajas
 - Método más eficiente
 - Cualquiera que sea el número de colisiones registradas en una posición, siempre será posible tratar una más.
- Desventajas
 - Espacio extra adicional que se requiere para la referencia a otros elementos.
 - El encadenamiento es menos complicada que la reasignación ya que si el arreglo llega a llenarse completamente, siempre es posible asignar más nodos y añadirlos a varias listas.
 - Si las listas llegan a ser muy largas no se va a obtener ninguno de los beneficios de la randomización o hashing.

Zona de desbordamiento

- Mantiene una zona reservada para aquellos elementos que llegan a colisionar.
- Cuando se produzca una colisión el elemento se va a localizar en esta zona de desbordamiento.
- Al realizar la búsqueda y comprobar si el elemento buscado está en la posición dada por su tabla hash, si esa posición ya está ocupada por otro elemento con el mismo valor de hashing, se seguirá buscando a partir del inicio de la zona de desbordamiento de manera secuencial.

Zona de desbordamiento

- Ejemplo:

Claves		Desbordamientos	
0	80	0	13
1		1	104
2	12	2	79
3	43	3	03
4	54	4	35
5	25	5	89
6	56	.	
7		.	
8		.	
9	99	n	

Conclusiones

- Las tablas Hash permite que el coste medio de las operaciones de insertar, buscar y eliminar sea constante.
 - siempre que el factor de carga (grado de ocupacion) no sea excesivo para reducir la probabilidad de colisión
- Hay que elegir correctamente la funcion de hash:
 - deber ser fácilmente calculable, sin costosas operaciones
 - debe tener una buena distribución de valores entre todas las componentes de la tabla.
- Ejemplos de aplicación de tablas de dispersión
 - corrector ortográfico
 - tablas de símbolos de un compilador
 - Los nombres símbolos (variables) son la clave, y los datos de símbolos (tipo, ubicación) están contenidos en el objeto almacenado.
 - diccionarios, etc.